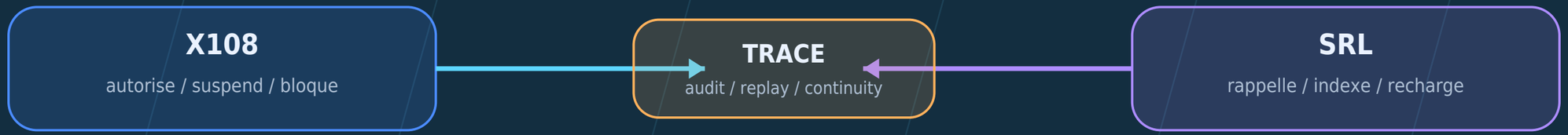
ATLAS VISUEL OBSIDIA / X108 + SRL

V3 - Schema Flow Edition

Gouvernance decisionnelle, preuve, publication maitrisee et memoire sessionnelle



Phrase verrou : X108 autorise. SRL rappelle. X108 gouverne l’action. SRL gouverne la continuite contextuelle.

Sources + contrat de fabrication

Deux documents fusionnes, une version schema-first compacte.

SOURCE 1

audit memoire lot(2).docx
P78, X108, domaines, preuve, exposition, synthese kernel

SOURCE 2

audit memoire ect(2).docx
SRL, memoire froide, fantome, session card, rappel, audit readonly

VERSION LONGUE

Detail complet, beaucoup de pages, texte source et instructions brutes.

VERSION V3

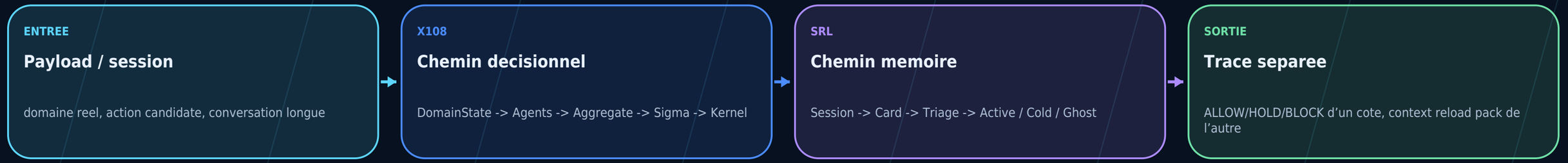
Autour de 100 pages max : flux visibles, schemas, matrices, listes compactes.

MESSAGE CLE

Reprendre les mots, formulations, listes et boundaries source. Ne pas inventer. Ne pas lisser.

Carte globale - deux chemins

Un flux gouverne l’action. Un flux garde la continuite.



MESSAGE CLE

X108 autorise. SRL rappelle. Les deux chemins se croisent par la trace, pas par l’autorite.

Legende visuelle

Codes graphiques utilises dans les pages.

AUTORITE

Kernel X108
seul a autoriser

SIGNAL

Agents / votes
observent et signalent

SURVEILLANCE

Sigma
stabilite et proof signals

PREUVE

OS3 / audit
hash / replay / attestation

MEMOIRE

SRL
fiche / index / reprise

BOUNDARY

readonly / no-write
KX108_ONLY / cannot ACT

MESSAGE CLE

Chaque page suit la logique : grand schema visuel, donnees obligatoires, message cle, erreur a eviter.

Plan V3 - 100 pages maximum

Version detaillee mais compressee en cartes, flux et matrices.

Bloc	Pages	Role visuel
Ouverture	1-5	sources, contrat, carte globale, legende
P78	6-17	classification documentaire public/prive/preuve/investisseur/demo/archive
Securite Kernel	18-27	BLOCK > HOLD > ALLOW, temporal lock, non-contournement, IR
Domaines	28-49	Banque, Trading, GPS, E-commerce, AgentVote
Machinerie commune	50-67	DomainAggregate, Meta-agents, Sigma, confidence, Envelope, Audit, OS
Lean / Socle formel	68-74	57 theorem families, bridge runtime, preuve visuelle finale
SRL	75-98	memoire froide/fantome, card, index, recall, boundaries readonly
Fusion finale	99-100	roles separes et carte finale fusionnee

MESSAGE CLE

Objectif : garder le detail structurant de la version longue sans produire un rapport de 300 pages.

PARTIE I - P78 / Classification documentaire

Support presentation != preuve technique. Public proof != moteur proprietaire complet.

Flux visibles

Details compresses

Boundaries lisibles

P78 - contexte valide avant split

Historique P56 -> P77 repris en timeline lisible.

P56-P61

Preuve + bus

Sigma veto-only, core inventory, import triage, audit tooling, tests proof, bus dry-run

P62-P66

Classification + SRL

77 fichiers lourds, fusion audit, ledger, OS adapters readonly, SRL readonly memory layer

P67-P72

Boundaries + graph

semantic split, auth/routes, filesystem, network egress, source packs, invariant graph

P73-P77

Stabilisation

agents readonly, Sigma safe, runtime risk, GPS reconciliation, canon wording cleanup

MESSAGE CLE

P78 arrive apres une chaine de paliers valides ; il ne modifie pas runtime/Sigma/routes/connectors.

P78 - commits connus

Repere historique conserve depuis la version longue.

01

P65 : 759b6d9 - OS adapters readonly/dry-run + bus registry

02

P66 : da51e45 - SRL readonly memory layer

03

P67 : 4a54438 - boundary semantic split

04

P68 : 754fc74 - API auth/route exposure

05

P69 : c72dea7 - filesystem/path exposure

06

P70 : 8d1bd3d - network egress/connectors

07

P71 : b5ab1da - source runtime/source packs

08

P72 : 1d61ebb - invariant graph/formal proof alignment

09

P73 : d7b22d8 - agents complementaires readonly

10

P74 : c526fa5 - Sigma safe evolution

11

P75 : ba2a154 - runtime core risk review

12

P76 : 7bd1019 - GPS terrain portable reconciliation

13

P77 : 3f80037 - canon wording targeted cleanup

MESSAGE CLE

Ces commits servent de trace contextuelle, pas de contenu public a exposer brut.

P78 - 13 questions a traiter

Ce que la matrice de classification doit repondre.

01
Quels fichiers sont des preuves techniques ?

02
Quels fichiers sont des docs d’audit ?

03
Quels fichiers sont des supports publics ?

04
Quels fichiers sont des supports investisseurs ?

05
Quels fichiers sont des narratifs / doctrine / pitch ?

06
Quels fichiers sont des demos ?

07
Quels fichiers sont des archives ?

08
Quels fichiers sont source packs locaux ?

09
Quels fichiers doivent rester prives/proprietaires ?

10
Quels fichiers peuvent etre publies ?

11
Quels fichiers ne doivent pas etre inclus dans un pack public ?

12
Quels fichiers peuvent nourrir un RSSI evidence pack ?

13
Quels fichiers doivent etre separes avant freeze P80 ?

MESSAGE CLE

P78 doit produire une matrice de classification et, si safe, creer uniquement des index docs.

P78 - surfaces a auditer

Dossiers et fichiers a cartographier.

01
docs/

02
docs/core_import/

03
docs/freeze/

04
docs/source_packs/

05
docs/public/

06
docs/investor/

07
docs/demo/

08
docs/proof/

09
specs/

10
specs/_invariant_graph/

11
README.md

12
LIMITS.md

13
KERNEL_OVERVIEW.md

14
GLOSSAIRE.md

15
_source_packs/

16
_tmp_core_import/

17
_freezes/

18
.local_audits/

19
audit/

20
proofs/

21
apps/obsidia_api/

22
runtime_wiring/

23
sigma/

24
periphery/

25
connectors/

MESSAGE CLE

Auditer ne veut pas dire deplacer massivement.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

P78 - ne pas toucher

Boundary stricte de non-mutation.

01
sigma/

02
runtime_wiring/

03
apps/obsidia_api/routes/

04
apps/obsidia_api/*.py

05
connectors/

06
periphery/

07
proofs/V18_3_1/

08
proofs/lean/

09
_tmp_core_import/

10
_source_packs/

11
_freezes/

12
.local_audits/

13
audit/world_action_bus.jsonl

14
proofs/PROOFKIT_REPORT.json

MESSAGE CLE

P78 ne doit pas supprimer, ne doit pas deplacer massivement, ne doit pas modifier runtime/Sigma/routes/connectors.

P78 - classifications

Statuts documentaires repris sans “etc.”.

01 DOC_PROOF_TECHNICAL	02 DOC_AUDIT_TRAIL	03 DOC_CORE_IMPORT_LEDGER
04 DOC_FORMAL_PROOF_MAP	05 DOC_RSSI_EVIDENCE_CANDIDATE	06 DOC_PUBLIC_SAFE
07 DOC_PUBLIC_NEEDS_SOFTENING	08 DOC_INVESTOR_NARRATIVE	09 DOC_DEMO_ONLY
10 DOC_PRIVATE_PROPRIETARY	11 DOC_SOURCE_PACK_LOCAL_ONLY	12 DOC_ARCHIVE_ONLY
13 DOC_RUNTIME_INTERNAL	14 DOC_API_INTERNAL	15 DOC_MEMORY_INTERNAL
16 DOC_CONNECTOR_INTERNAL	17 DOC_DO_NOT_PUBLISH	18 DOC_UNKNOWN_REVIEW

MESSAGE CLE

Les categories ne sont pas interchangeables.

P78 - decisions possibles

Actions de classification, pas actions runtime.

01
KEEP_AS_TECHNICAL_PROOF

02
KEEP_AS_AUDIT_LEDGER

03
KEEP_AS_PUBLIC_SAFE

04
MOVE_LATER_PUBLIC

05
MOVE_LATER_INVESTOR

06
MOVE_LATER_DEMO

07
KEEP_PRIVATE_PROPRIETARY

08
KEEP_ARCHIVE_ONLY

09
KEEP_SOURCE_PACK_LOCAL_ONLY

10
DO_NOT_PUBLISH

11
REQUIRES_REVIEW

MESSAGE CLE

La sortie P78 est un index documentaire, pas une mutation moteur.

P78 - regles de separation

Preuve, public, investisseur, demo, archive et local-only.

Objet A	!=	Objet B	Raison
Support presentation	!=	preuve technique	un support explique ; une preuve lie tests, commits, manifests, hashes
Public proof	!=	moteur proprietaire complet	publier sans exposer le coeur proprietaire
Source pack local	!=	canon public	local-only tant que non canonise
Narratif investisseur	!=	audit RSSI	ne doit pas pretendre plus que P72/P77 autorise
Demo	!=	runtime souverain	demontrer sans donner autorite
Archive	!=	runtime load	archive reste archive, pas runtime

MESSAGE CLE

Le but est de publier sans exposer le moteur proprietaire et sans melanger preuve, pitch et demo.

P78 - sorties a creer

Index docs + test. Scripts optionnels si safe.

SORTIES PRINCIPALES

docs/core_import/P78_PRESENTATION_PROOF_PUBLIC_PRIVATE_SPLIT.md
docs/core_import/P78_PRESENTATION_PROOF_PUBLIC_PRIVATE_SPLIT.json
tests/test_p78_presentation_proof_public_private_split.py

OPTIONNEL

scripts/audit_presentation_proof_public_private_split_p78.py
docs/public/README_PUBLIC_BOUNDARY.md
docs/investor/README_INVESTOR_BOUNDARY.md
docs/proof/README_PROOF_BOUNDARY.md
docs/demo/README_DEMO_BOUNDARY.md

MESSAGE CLE

Mode attendu : AUDIT_AND_DOC_INDEX_ONLY.

P78 - JSON attendu / statut et split model

Extrait structurel du JSON attendu.


```
{
"audit_id": "P78",
"status": "P78_PRESENTATION_PROOF_PUBLIC_PRIVATE_SPLIT_READY",
"mode": "AUDIT_AND_DOC_INDEX_ONLY",
"source_patch_applied": false,
"files_imported_count": 0,
"split_decision": "CLASSIFY_AND_INDEX_ONLY",
"split_model": {
"technical_proof": "", "audit_trail": "",
"formal_proof_map": "", "rssi_evidence_candidate": "",
"public_safe": "", "public_needs_softening": "",
"investor_narrative": "", "demo_only": "",
"private_proprietary": "", "source_pack_local_only": "",
"archive_only": "", "do_not_publish": ""
},
"files_scanned_count": 0,
"doc_matrix": [], "category_counts": {}, "decision_counts": {}
}
```


MESSAGE CLE

Le JSON prouve que P78 classe et indexe seulement.

P78 - JSON attendu / flags de non-mutation

Tous les verrous doivent rester false.


```
  "runtime_modified": false,
  "sigma_modified": false,
  "routes_modified": false,
  "srl_modified": false,
  "connectors_modified": false,
  "source_packs_modified": false,
  "proofs_modified": false,
  "lean_proofs_modified": false,
  "act_enabled": false,
  "memory_write_enabled": false,
  "graphiti_write_enabled": false,
  "neo4j_write_enabled": false,
  "kernel_mutation_enabled": false,
  "x108_merge_enabled": false,
  "next_step": "P79_RSSI_EVIDENCE_PACK_AND_GITHUB_SECURITY_AUDIT"
```


MESSAGE CLE

Aucune activation runtime, aucune ecriture memoire, aucun merge X108.

PARTIE II - Securite / Gouvernance du Kernel

X108 n’est pas un simple score. C’est un verrou de gouvernance.

Flux visibles

Details compresses

Boundaries lisibles

Schema 05 - Priorite BLOCK > HOLD > ALLOW

Pourquoi ALLOW n’est jamais le defaut ?



A NE PAS FAIRE

Ne pas representer les trois etats comme equivalents. Mauvais : ALLOW / HOLD / BLOCK. Bon : BLOCK > HOLD > ALLOW.

MESSAGE CLE

ALLOW arrive seulement quand BLOCK et HOLD n’ont rien trouve.

Schema 05 - regles a afficher

Seuils et conditions souveraines.

BLOCK

contradictions >= 2 -> BLOCK
FRAUD_PATTERN -> BLOCK

HOLD

unknowns > 1 -> HOLD
confidence < 0.45 -> HOLD
risk_flags >= 2 + confidence < 0.72 -> HOLD

ALLOW

sinon -> ALLOW
seulement si aucun BLOCK et aucun HOLD

LECTURE

La hierarchie est souveraine. Ce n’est pas une moyenne.

MESSAGE CLE

Un bon score metier ne force jamais ALLOW. Une seule condition critique peut bloquer.

Schema 06 - HOLD / Temporal Lock

Pourquoi le kernel suspend au lieu d’autoriser ?

ACTION CANDIDATE

demande d’action possible

↓

X108 CHECK

elapsed_s, min_required_elapsed_s, confidence, unknowns, risk_flags, sigma_report

↓

CONDITION NON SATISFAITE

temps insuffisant, contexte incomplet, trajectoire instable, confiance basse, preuve incomplete

↓

HOLD

action non autorisee maintenant, aucune execution reelle, attente / contexte / reevaluation

DECLENCHEURS

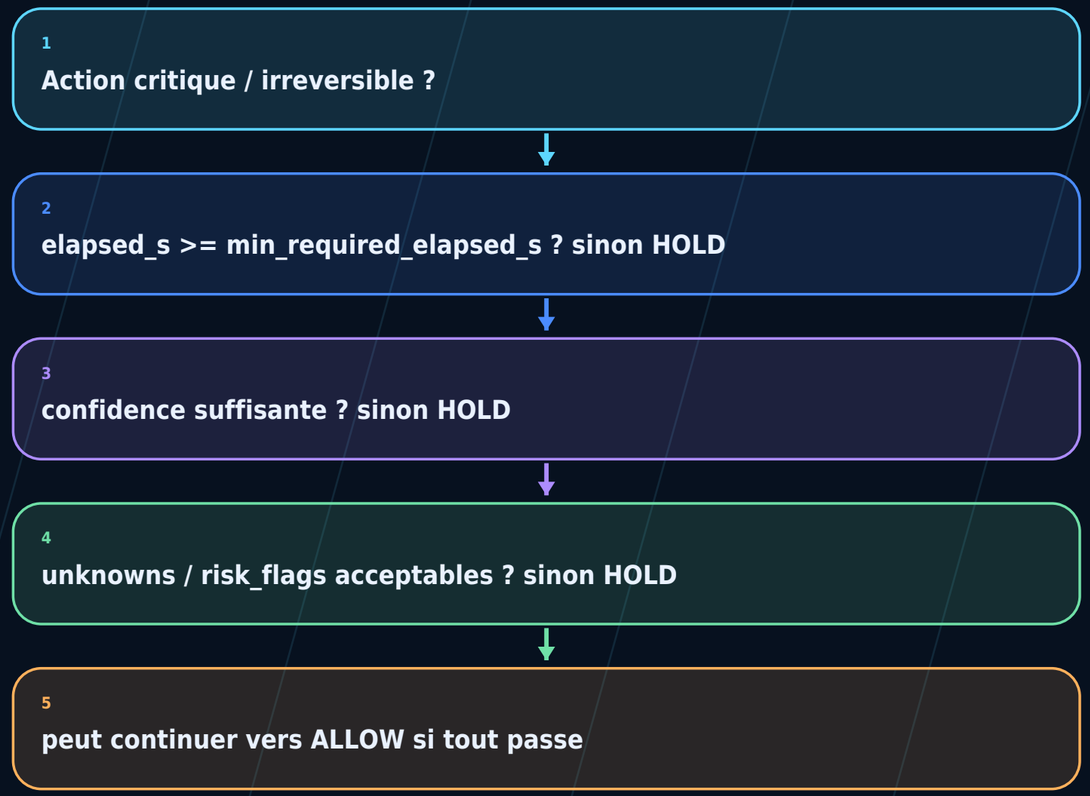
unknowns > 1 ; confidence < 0.45 ; risk_flags >= 2 + confidence < 0.72 ; elapsed_s < min_required_elapsed_s ; stabilite insuffisante ; preuve/attestation/trace insuffisante.

MESSAGE CLE

HOLD est une decision active, pas une absence de decision.

Schema 06 - arbre de decision temporal

Action possible ne veut pas dire action autorisee.



A NE PAS DIRE

Ne pas presenter HOLD comme erreur, echec, indecision ou bug.

DIRE

HOLD = verrou de securite, delai actif, refus temporaire d’action reelle.

MESSAGE CLE

X108 peut suspendre pour attendre, verifier, completer ou rejouer.

Schema 07 - Non-contournement

Qui a le droit d’autoriser une action ?

Couche	Droit d’autoriser ACT ?	Role reel
Agents domaine	NON	peuvent signaler
AgentVote	NON	signal atomique
DomainAggregate	NON	synthese metier
Meta-agents	NON	controles transverses
Sigma	NON	peut alerter
Projection francaise	NON	peut exposer
Terminal / API	NON	interface
Memoire / contexte	NON	contexte readonly
Audit / replay	NON	preuve apres coup
Kernel X108	OUI	ALLOW / HOLD / BLOCK

MESSAGE CLE

Tout peut signaler. Seul X108 autorise.

Schema 07 - flux no-bypass

Toutes les couches avant X108 produisent des signaux, pas des ACT.



BOUNDARIES

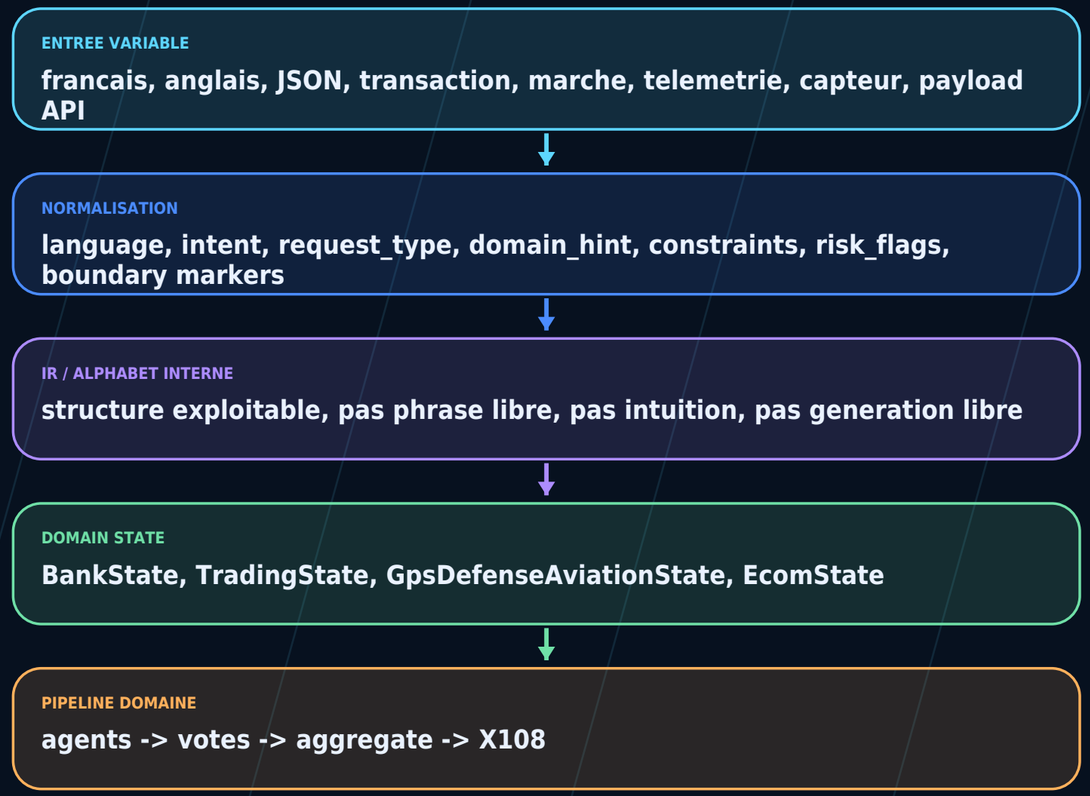
decision_authority = KX108_ONLY
emits_act = false
memory_write = false
readonly = true
no-bypass / no-write

MESSAGE CLE

decision_authority = KX108_ONLY doit rester visible.

Schema 08 - Entree variable -> IR / alphabet interne

Comment traiter plusieurs langues, payloads et signaux terrain ?



MESSAGE CLE

La langue est variable. L’alphabet interne est stable.

Schema 08 - formulations correctes

Compréhension structurelle sans parler de LLM.

A NE PAS DIRE

le kernel parle francais
le kernel est un LLM
le kernel comprend n’importe quoi directement

DIRE

l’entree est normalisee
l’entree devient structure
la structure devient gouvernable

MESSAGE CLE

Le kernel ne comprend pas le francais comme un chatbot. Il recoit une structure gouvernable.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Mini-checklist du lot 2

Validation et ordre recommande.

Schema	Valide si
05	BLOCK apparait avant HOLD ; HOLD avant ALLOW ; ALLOW dernier cas ; seuils presents
06	HOLD est decision active ; elapsed_s / min_required_elapsed_s visibles ; aucune action reelle ne sort pendant HOLD
07	agents, aggregate, meta-agents, Sigma, projection = cannot ACT ; Kernel X108 = seule autorite ; KX108_ONLY visible
08	francais / anglais / JSON / capteur / transaction visibles ; IR central ; DomainState sortie ; pas de LLM / pas de generation libre
Ordre	07 Non-contournement -> 05 Priorite -> 06 Temporal lock -> 08 Entree structurelle

MESSAGE CLE

On fixe d’abord l’autorite, puis la priorite, puis le frein temporel, puis l’entree structurelle.

PARTIE III - Domaines principaux

Banque, Trading, GPS et E-commerce comme etats metiers gouvernes.

Flux visibles

Details compresses

Boundaries lisibles

Vue comparative des domaines

Les domaines ne vont pas directement vers la decision.

BANQUE

transaction / fraude / identite / policy / solvabilite

TRADING

marche / signal / volatilite / risque / execution

GPS / DEFENSE / AVIATION

capteurs / trajectoire / source conflict / time skew / brownout / attestation

E-COMMERCE

domaine annexe, extension possible, pas coeur prioritaire

MESSAGE CLE

Le domaine propose. Les agents signalent. Sigma surveille. X108 decide. OS3 prouve.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 09 - Banque : flux complet

Que devient une transaction bancaire quand elle entre dans Obsidia / X108 ?



MESSAGE CLE

Ne pas confondre BLOCK metier avec BLOCK kernel.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 09 - payload bancaire

Tous les champs obligatoires a afficher.

01
transaction_type

02
channel

03
amount

04
account_balance

05
counterparty_known

06
counterparty_age_days

07
behavior_shift_score

08
fraud_score

09
policy_limit

10
affordability_score

11
urgency_score

12
identity_mismatch_score

13
narrative_conflict_score

14
recovery_path_score

15
proof_ready

16
elapsed_s

17
min_required_elapsed_s

MESSAGE CLE

Le payload bancaire devient BankState avant tout vote, agregation ou decision.

Schema 09 - 12 agents banque

Les agents banque signalent, ils ne decident pas.

01
TransactionContextAgent

02
CounterpartyAgent

03
LiquidityExposureAgent

04
BehaviorShiftAgent

05
FraudPatternAgent

06
LimitPolicyAgent

07
AffordabilityAgent

08
TemporalUrgencyAgent

09
IdentityMismatchAgent

10
NarrativeConflictAgent

11
RecoveryPathAgent

12
BankProofAgent

MESSAGE CLE

Chaque agent lit une facette du risque bancaire.

Schema 09 - AgentVote et aggregate_bank()

Le verdict metier candidat n’est pas encore autorite.

AGENTVOTE[]

claim
confidence
severity_hint
unknowns
contradictions
...

AGGREGATE_BANK()

scores AUTHORIZE / ANALYZE / BLOCK
confidence
unknowns
contradictions
risk_flags
...

MESSAGE CLE

BankState -> agents banque -> AgentVote[] -> aggregate_bank -> candidat -> X108.

Schema 10 - Banque : metriques et cas X108

Quels cas bancaires activent le mur X108 ?

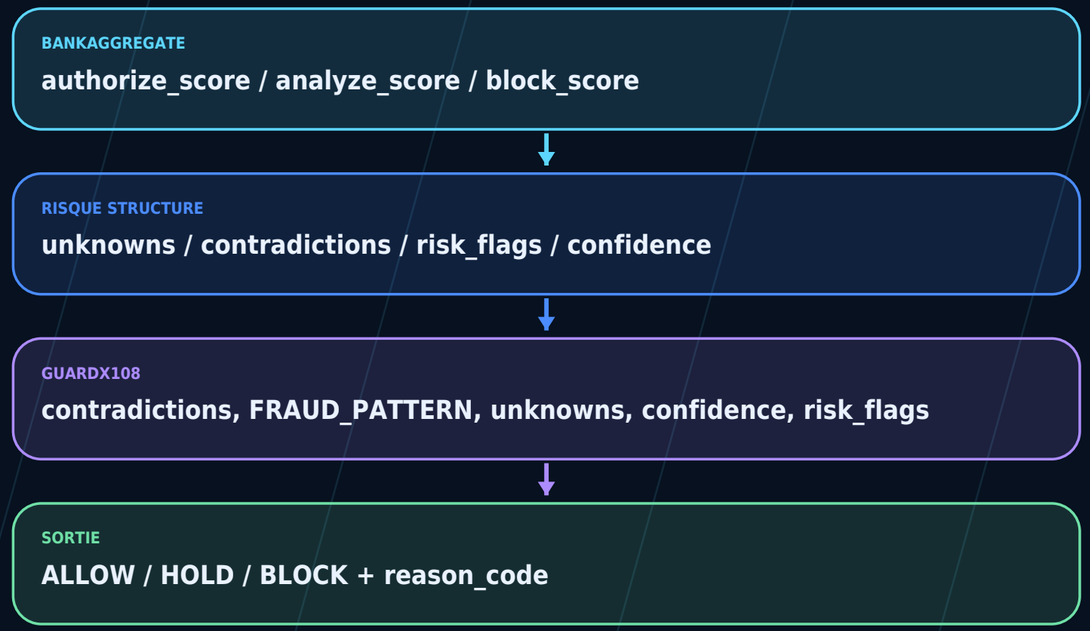
Cas banque	Signals / metriques	Verdict metier candidat	Sortie X108
Transaction coherente	confidence haute	AUTHORIZE	ALLOW
Contrepartie inconnue	unknowns	ANALYZE	HOLD
Solde / liquidite fragile	affordability / liquidity	ANALYZE	HOLD
Fraude forte	FRAUD_PATTERN / fraud_score	BLOCK	BLOCK
Identite incoherente	identity_mismatch_score	ANALYZE / BLOCK	HOLD ou BLOCK selon signaux
Contradictions multiples	contradictions >= 2	BLOCK	BLOCK
Confiance trop basse	confidence < 0.45	ANALYZE	HOLD
Risques multiples	risk_flags >= 2 + confidence < 0.72	ANALYZE	HOLD

MESSAGE CLE

AUTHORIZE n’est pas ALLOW. ANALYZE n’est pas HOLD. Le domaine parle metier. X108 parle autorite.

Schema 10 - mur banque

Le mur X108 lit les signaux agreges, pas une intention libre.



MESSAGE CLE

AUTHORIZE = verdict metier candidat. ALLOW = autorisation kernel.

Schema 11 - Trading / Bourse : flux complet

Que devient un signal de marche dans Obsidia / X108 ?



MESSAGE CLE

Un signal de marche peut proposer. Il ne peut pas autoriser l’execution.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 11 - payload marche

Tous les champs de marche a afficher.

01
symbol

02
prices

03
highs

04
lows

05
volumes

06
spreads_bps

07
event_risk_scores

08
sentiment_scores

09
predicted_returns

10
exposure

11
drawdown

12
order_book_imbalance

13
execution_cost_bps

MESSAGE CLE

Le signal marche devient TradingState avant toute execution.

Schema 11 - 17 agents trading

Les agents trading signalent, ils ne decident pas.

01
MarketDataAgent

02
LiquidityAgent

03
VolatilityAgent

04
MacroAgent

05
CorrelationAgent

06
EventAgent

07
MomentumAgent

08
MeanReversionAgent

09
BreakoutAgent

10
PatternAgent

11
SentimentAgent

12
PredictionAgent

13
PortfolioAgent

14
ExecutionQualityAgent

15
RegimeShiftAgent

16
PortfolioStressAgent

17
ProofConsistencyAgent

MESSAGE CLE

Chaque agent lit une facette du risque marche, portefeuille, execution ou preuve.

Schema 12 - Trading : metriques et cas X108

Quels cas trading declenchent HOLD ou BLOCK ?

Cas trading	Signal / metrique	Risque X108	Sortie
Signal coherent	confidence haute + proof consistency	chemin stable	ALLOW possible
Volatilite excessive	volatility spike	risque instable	HOLD
Regime instable	RegimeShiftAgent	flip brutal / trajectoire instable	HOLD ou BLOCK
Contradiction BUY / SELL	contradictions	conflit directionnel	HOLD ou BLOCK
Exposition trop forte	exposure	risque portefeuille	HOLD
Drawdown	drawdown	perte / stress	HOLD
Event risk	event_risk_scores	evenement non classifie	HOLD
Cout execution trop eleve	execution_cost_bps	slippage / qualite execution	HOLD
Proof consistency insuffisant	ProofConsistencyAgent	preuve incomplete	HOLD

MESSAGE CLE

Le market_verdict candidat ne vaut pas autorisation. X108 tranche la sortie reelle.

Schema 12 - mur trading

Proposer un trade n’est pas l’executer.

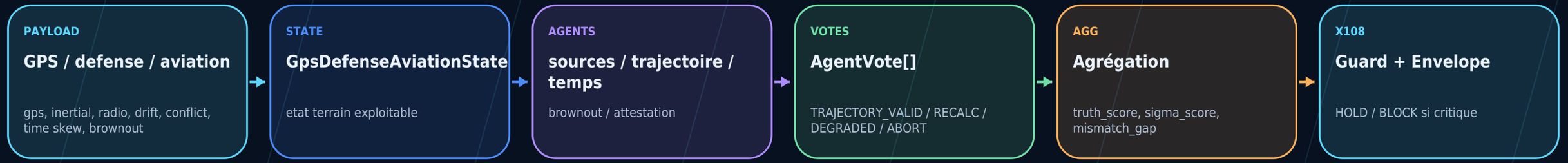


MESSAGE CLE

Trading parle direction et risque. X108 parle droit d’agir.

Schema 13 - GPS / Defense / Aviation : flux complet

Que devient une trajectoire ou un signal critique terrain ?



MESSAGE CLE

Si les sources se contredisent ou si la trajectoire n’est pas stable, le systeme ne laisse pas passer l’action critique.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 13 - GPS : payload et agents

Champs et agents du domaine GPS / Defense / Aviation.

01
gps_available

02
inertial_available

03
radio_available

04
trajectory_drift_score

05
source_conflict_score

06
time_skew_score

07
brownout_score

08
attestation_ready

09
rollback_possible

10
SourceAvailabilityAgent

11
TrajectoryIntegrityAgent

12
SourceConflictAgent

13
TimeSkewAgent

14
BrownoutAgent

15
AttestationReadinessAgent

MESSAGE CLE

Le domaine GPS compare sources, trajectoire, temps, energie, attestation et rollback.

Schema 14 - GPS : metriques et cas X108

Quels signaux terrain activent HOLD ou BLOCK ?

Cas GPS	Signal / metrique	Verdict candidat	Sortie X108
Sources coherentes	gps / inertial / radio alignes	TRAJECTORY_VALID	ALLOW possible
Source conflict eleve	source_conflict_score	DEGRADED_NAVIGATION	HOLD / BLOCK
Trajectory drift fort	trajectory_drift_score	RECALC / DEGRADED	HOLD
Time skew eleve	time_skew_score	DEGRADED_NAVIGATION	HOLD
Brownout eleve	brownout_score	ABORT ou DEGRADED	HOLD / BLOCK
Attestation non prete	attestation_ready false	RECALC / DEGRADED	HOLD
Rollback impossible	rollback_possible false	ABORT_TRAJECTORY	BLOCK ou HOLD selon criticite

MESSAGE CLE

Le systeme compare GPS, inertiel, radio, trajectoire, temps, energie, attestation et rollback.

Schema 14 - conflit source terrain

La trajectoire n’est valide que si les sources restent coherentes.



MESSAGE CLE

GPS ok / GPS pas ok est trop pauvre : le systeme compare plusieurs sources.

Schema 15 - E-commerce : annexe domaine

Extension possible, pas coeur prioritaire.



MESSAGE CLE

E-commerce sert a montrer l’extensibilite domaine sans prendre la place Banque / Trading / GPS.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 15 - statut e-commerce

A montrer comme extension, pas comme coeur.

STATUT

Domaine annexe. Extension possible. Pas coeur prioritaire.

A EVITER

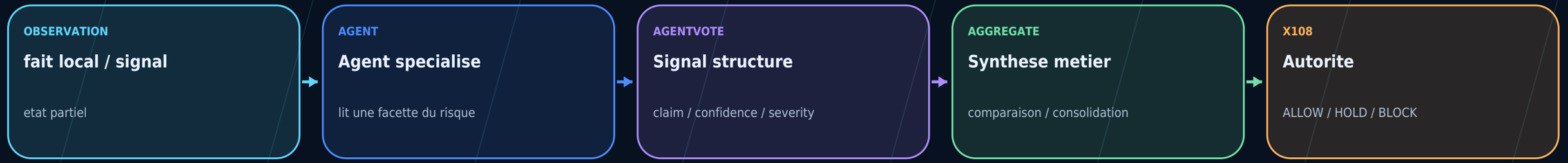
Ne pas mettre E-commerce au meme niveau strategique que Banque / Trading / GPS dans le coeur de preuve.

MESSAGE CLE

L’annexe e-commerce prouve l’extension domaine sans diluer les preuves prioritaires.

Schema 16 - AgentVote : unite atomique commune

Comment chaque agent transforme une observation metier en signal structure comparable ?



MESSAGE CLE

AgentVote = signal atomique. DomainAggregate = synthese metier. X108 = autorite finale.

Schema 16 - champs communs AgentVote

Sans ce schema, les domaines ressemblent a des listes d’agents.

01
claim

02
confidence

03
severity_hint

04
unknowns

05
contradictions

06
risk_flags

07
proposed_verdict

08
evidence_refs

MESSAGE CLE

Avec AgentVote, chaque observation devient comparable.

Mini-checklist lots 3 + 4

Validation des domaines et des signaux communs.

Schema	Validation
09 Banque	Payload -> BankState -> 12 agents -> AgentVote[] -> aggregate_bank -> candidat -> X108 -> Envelope -> Audit
10 Banque cas	AUTHORIZE != ALLOW ; ANALYZE != HOLD ; BLOCK metier != BLOCK kernel
11 Trading	TradingState avant execution ; 17 agents visibles ; market_verdict candidat visible
12 Trading cas	volatilite, regime, contradictions, exposure, drawdown, event risk, execution cost, proof consistency
13 GPS	payload + agents + votes + aggregation + Guard ; sources comparees
14 GPS cas	source conflict, drift, time skew, brownout, attestation, rollback
15 E-commerce	extension annexe, pas coeur prioritaire
16 AgentVote	signal atomique commun visible avant DomainAggregate

MESSAGE CLE

Les domaines produisent des etats metiers gouvernables, pas une autorisation directe.

PARTIE IV - Machinerie commune

AgentVote, DomainAggregate, Meta-agents, Sigma, triple confidence, Envelope, Audit.

Flux visibles

Details compresses

Boundaries lisibles

Schema 17 - DomainAggregate

Comment plusieurs AgentVote deviennent une structure exploitable par X108 ?



MESSAGE CLE

Le verdict metier candidat n’est pas une decision kernel.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 17 - fonctions d’agregation

Une fonction par domaine, une sortie commune.

AGGREGATE_BANK()

AUTHORIZE / ANALYZE / BLOCK

AGGREGATE_TRADING()

BUY / SELL / HOLD

AGGREGATE_GPS_DEFENSE_AVIATION()

VALID / RECALC / DEGRADED / ABORT

AGGREGATE_ECOM()

verdict metier candidat

MESSAGE CLE

Chaque domaine parle metier. X108 parle autorite.

Schema 18 - Meta-agents

Qu’est-ce qui securise la decision entre verdict metier et X108 ?



MESSAGE CLE

Les meta-agents securisent le passage entre metier et autorite, mais ne deviennent pas autorite.

Schema 18 - meta-agents transverses

Controles que les agents domaine ne couvrent pas directement.

01

UnknownsAgent

02

ConflictResolutionAgent

03

PolicyScopeAgent

04

TicketReadinessAgent

05

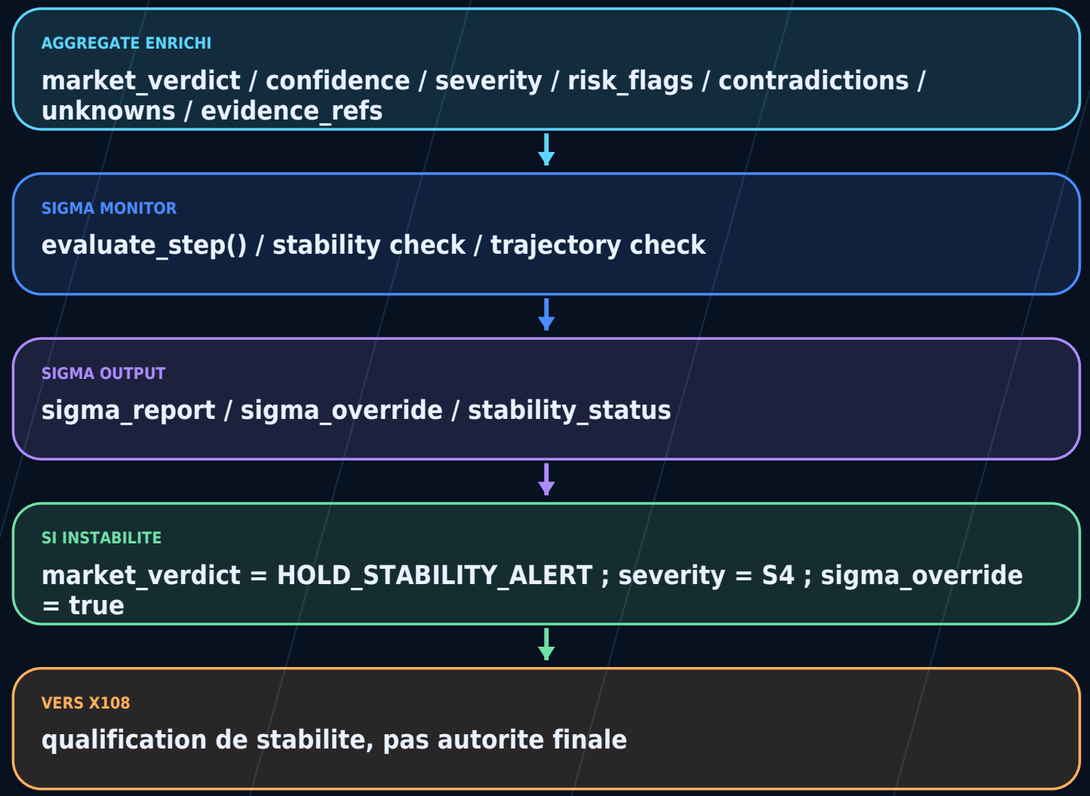
TraceIntegrityAgent

MESSAGE CLE

Preuve, trace, ticket, policy, attestation, replay, coherence, override humain, severite.

Schema 19 - Sigma / proof signals

Ou intervient la stabilite dynamique ?



MESSAGE CLE

Sigma surveille. X108 decide.

Schema 19 - role Sigma

Qualification de stabilite, pas autorite finale.

SIGMA PEUT

evaluer stabilite
qualifier trajectoire
produire sigma_report
alerter via sigma_override

SIGMA NE PEUT PAS

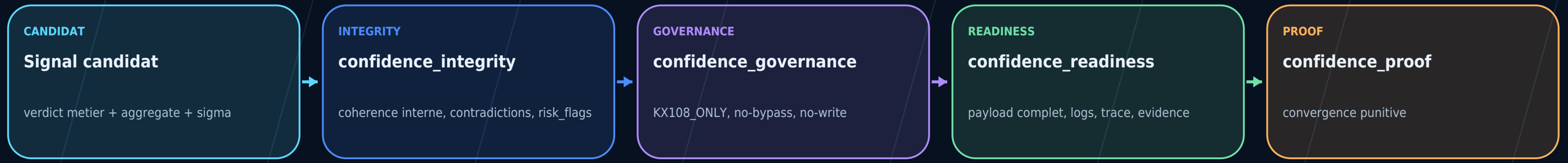
autoriser ACT
remplacer GuardX108
redonner ALLOW
modifier le noyau

MESSAGE CLE

Ne pas mettre Sigma apres X108 comme s’il pouvait revalider.

Schema 20 - Triple confidence

Pourquoi un bon score local ne suffit pas ?



MESSAGE CLE

Le systeme ne laisse pas une seule dimension imposer ALLOW.

Schema 20 - detail des dimensions

La confiance unique ne suffit pas.

CONFIDENCE_INTEGRITY

coherence interne / contradictions / risk_flags / stabilite du verdict / coherence des champs

CONFIDENCE_GOVERNANCE

decision_authority = KX108_ONLY / no-bypass / no-write / GuardX108 respecte / Sigma non souverain / projection readonly

CONFIDENCE_READINESS

payload complet / runtime lisible / sigma_report present / logs presents / trace disponible / evidence_refs exploitables

CONFIDENCE_PROOF

convergence punitive : si une dimension tombe, tout chute

MESSAGE CLE

Convergence punitive : si une dimension tombe, tout chute.

Schema 21 - Canonical Decision Envelope

Sous quelle forme sort une decision X108 ?



MESSAGE CLE

La sortie X108 est un objet de decision, pas une prose.

Schema 21 - champs Envelope

Objet de decision canonique.

01
domain

02
market_verdict

03
confidence

04
contradictions

05
unknowns

06
x108_gate

07
reason_code

08
severity

09
decision_id

10
trace_id

11
attestation_ref

12
evidence_refs

MESSAGE CLE

Le kernel ne sort pas une phrase libre ; il sort un objet structure.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 22 - Audit / hash / replay

Comment rendre la decision verifiable ?



MESSAGE CLE

Audit / hash / replay ne decident pas. Ils prouvent et verifient.

Schema 22 - trace de preuve

OS3 produit trace, enveloppe, ticket et preuve.

PREUVE

trace
ticket
hash
attestation
evidence_refs

REPLAY

decision rejouable
verification de coherence
controle du reason_code
inspection des signaux

MESSAGE CLE

L’audit prouve apres coup ; il ne devient jamais autorite.

Schema 23 - Exposition humaine readonly

Comment exposer la decision sans redecider ?



MESSAGE CLE

L’exposition humaine est readonly.

Schema 23 - boundaries exposition

La couche humaine comprend, expose, explique.

PEUT

projeter en francais
montrer ALLOW/HOLD/BLOCK
expliquer raison et gravite
montrer trace

NE PEUT PAS

changer la decision
autoriser ACT
contourner X108
modifier evidence

MESSAGE CLE

La projection francaise peut exposer. Elle ne peut pas autoriser.

Schema 24 - OS / Runtime / Bus

Quelles couches portent la machination ?

OS4
INTERFACE / EXPOSITION READONLY
payload / API / terminal / connecteur terrain / JSON / logs / projection FR

OS1
STRUCTURE / PARSE / VALIDATION
parse / validate / DomainState / action candidate / contract checks

OS2
METRIQUES / SCORES / SEUILS
confidence / scores metier / thresholds / risk_flags / unknowns / contradictions / ACT-HOLD signals

SIGMA
PROOF SIGNALS
stability / integrity / governance / readiness / sigma_report / sigma_override

OS0
KERNEL X108 / GUARD
regles / invariants / seuils / temporal lock / BLOCK > HOLD > ALLOW

OS3
AUDIT / PREUVE / REPLAY
CanonicalDecisionEnvelope / trace / ticket / hash / replay

MESSAGE CLE

OS4 expose. OS1 structure. OS2 mesure. Sigma surveille. OS0 decide. OS3 prouve.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 24 - couches a ne pas melanger

Montrer les couches qui portent le kernel.

STRUCTURE PORTEE

OS4 expose
OS1 structure
OS2 mesure
Sigma surveille
OS0 decide
OS3 prouve

A NE PAS MELANGER

UI privee
historique des repos
routes internes non publiees
source packs proprietaires

MESSAGE CLE

Ne pas melanger avec UI privee ou historique des repos.

Mini-checklist lot 5 + 6

Validation machinerie commune, sortie et preuve.

Schema	Validation
17 DomainAggregate	AgentVote[] -> aggregate_* -> DomainAggregate -> verdict metier candidat -> X108
18 Meta-agents	unknowns/conflict/policy/ticket/trace visibles ; ne deviennent pas autorite
19 Sigma	stabilite dynamique visible ; Sigma surveille ; X108 decide
20 Triple confidence	integrity/governance/readiness/proof visibles ; une dimension ne force pas ALLOW
21 Envelope	sortie canonique structuree, pas prose
22 Audit/hash/replay	prouve et verifie ; ne decide pas
23 Exposition readonly	projection humaine expose sans redecider
24 OS/runtime/bus	OS4 expose, OS1 structure, OS2 mesure, Sigma surveille, OS0 decide, OS3 prouve

MESSAGE CLE

La machinerie commune transforme les votes en preuve auditable sans court-circuiter X108.

PARTIE V - Socle formel et synthese kernel

Lean 57 et preuve visuelle finale du kernel.

Flux visibles

Details compresses

Boundaries lisibles

Schema 25 - Lean 57 theorem families

Qu’est-ce qui prouve les lois du kernel ?

01

Basic - 11 : seuils / ordres / proprietes de base / fondations logiques

02

TemporalKernel - 5 : attente X108 / temporalite / non-anticipation / pas d’ACT avant condition

03

TemporalBridge - 2 : pont temporalite <-> decision / coherence temps et gate

04

Refinement - 4 : passage structure -> decision / coherence abstraction-runtime

05

Consensus - 10 : agregation / priorite collective

06

ConsensusScratch - 2 : scratch consensus / exploration formelle

07

Merkle - 2 : evidence_ref / hash / audit_chain

08

CryptoAssumptions - 10 : attestation / signature / anti-replay

09

Seal - 1 : scellement de decision

10

Sensitivity - 3 : robustesse des seuils

11

SystemModel - 7 : transition / auditabilite / coherence globale

MESSAGE CLE

Le kernel n’est pas seulement un pipeline logiciel. Il est adosse a un socle formel.

Schema 25 - repartition 57

Le socle formel couvre temporalite, consensus, preuve et systeme.

Famille	Nombre	Couvre
Basic	11	seuils, ordres, proprietes de base, fondations logiques
TemporalKernel	5	attente X108, temporalite, non-anticipation, pas d’ACT avant condition
TemporalBridge	2	pont temps-decision, coherence entre temps et gate
Refinement	4	raffinement decisionnel et coherence abstraction-runtime
Consensus	10	agregation et priorite collective
ConsensusScratch	2	exploration formelle
Merkle	2	evidence_ref, hash, audit_chain
CryptoAssumptions	10	attestation, signature, anti-replay
Seal	1	scellement de decision
Sensitivity	3	robustesse des seuils
SystemModel	7	transition, auditabilite, coherence globale

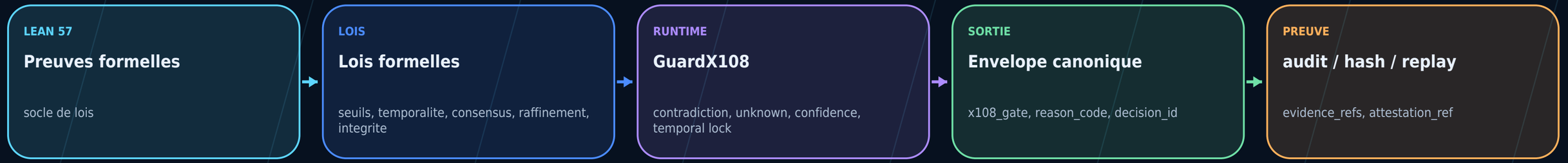
MESSAGE CLE

Lean doit etre relie aux lois runtime, sinon il reste une annexe isolee.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 26 - Lean -> Runtime Kernel

Comment les preuves Lean se relient au runtime X108 ?



MESSAGE CLE

La preuve formelle ne remplace pas le runtime. Elle encadre les lois que le runtime applique.

Schema 26 - lois runtime encadrees

Le runtime applique les lois, il ne les invente pas.

RUNTIME KERNEL

GuardX108
contradiction check
unknown check
confidence check
temporal lock
BLOCK > HOLD > ALLOW

SORTIE CANONIQUE

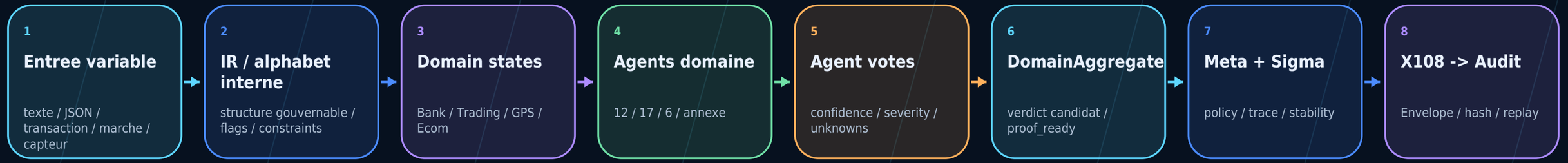
CanonicalDecisionEnvelope
x108_gate
reason_code
severity
decision_id
trace_id

MESSAGE CLE

Sans ce schema, Lean reste une annexe isolée.

Schema 27 - Synthese finale : preuve visuelle du kernel

Quelle est la preuve visuelle complete du kernel ?



MESSAGE CLE

Le domaine propose. Les agents signalent. Sigma surveille. X108 decide. OS3 prouve.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Schema 27 - erreurs a eviter dans la synthese

Le dernier schema doit expliquer tout le systeme en une image.

VISIBLE

entree variable
IR stable
domain states
agents
aggregate
...

A EVITER

DomainState -> ALLOW
Sigma autorite
audit decide
projection francaise decisionnelle
SRL qui modifie X108

MESSAGE CLE

La preuve visuelle finale doit rendre visible la chaine, pas seulement lister les composants.

PARTIE VI - SRL / Session Registry Layer

Continuite sans surcharge, memoire sans bruit, rappel sans relecture totale.

Flux visibles

Details compresses

Boundaries lisibles

SRL-01 - Memoire froide / fantome

Ne pas tout mettre dans le contexte actif, mais ne rien perdre structurellement.



MESSAGE CLE

FANTOME != supprime. FANTOME = hors contexte actif, mais retrouvable.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-01 - lois memoire

Le moteur ne doit pas oublier. Il doit reduire la charge contextuelle.

SUPPRESSION INTERDITE

ne rien perdre structurellement

MISE EN FROID AUTORISEE

reduire la charge contextuelle

REACTIVATION TRACABLE

un fragment revient sous condition et avec trace

MESSAGE CLE

Suppression interdite, mise en froid autorisee, reactivation tracable.

SRL-02 - Session card / fiche de reprise

Chaque session devient une fiche structuree, pas un bloc perdu dans l’historique.



MESSAGE CLE

Une session ne doit jamais mourir dans l’historique. Elle doit etre transformee en fiche de reprise.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-08 - Format canonique fiche session

Champs obligatoires de la fiche SRL.

01 session_id	02 date	03 session_type	04 project
05 main_topic	06 entry_context	07 previous_session_ref	08 starting_point
09 user_intent	10 exit_point	11 last_stable_point	12 next_action
13 unresolved_threads	14 active_blocks	15 decisions	16 validated_items
17 open_threads	18 frictions	19 pepites	20 files_links
21 commands_or_actions	22 cold_fragments	23 ghost_fragments	24 reactivation_rules
25 status			

MESSAGE CLE

La fiche contient decisions, pepites, frictions, fichiers, commandes, fragments froids et fantomes.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-03 - Positionnement SRL

La SRL est peripherique, non moteur, non decisionnelle.

CE QUE SRL EST

couche peripherique de continuite sessionnelle
range
indexe
classe
retrouve
...

CE QUE SRL N'EST PAS

pas memoire centrale Obsidia
pas Kernel
pas X108
pas decision
pas nouvelle couche interne OS0-OS4
pas runtime moteur

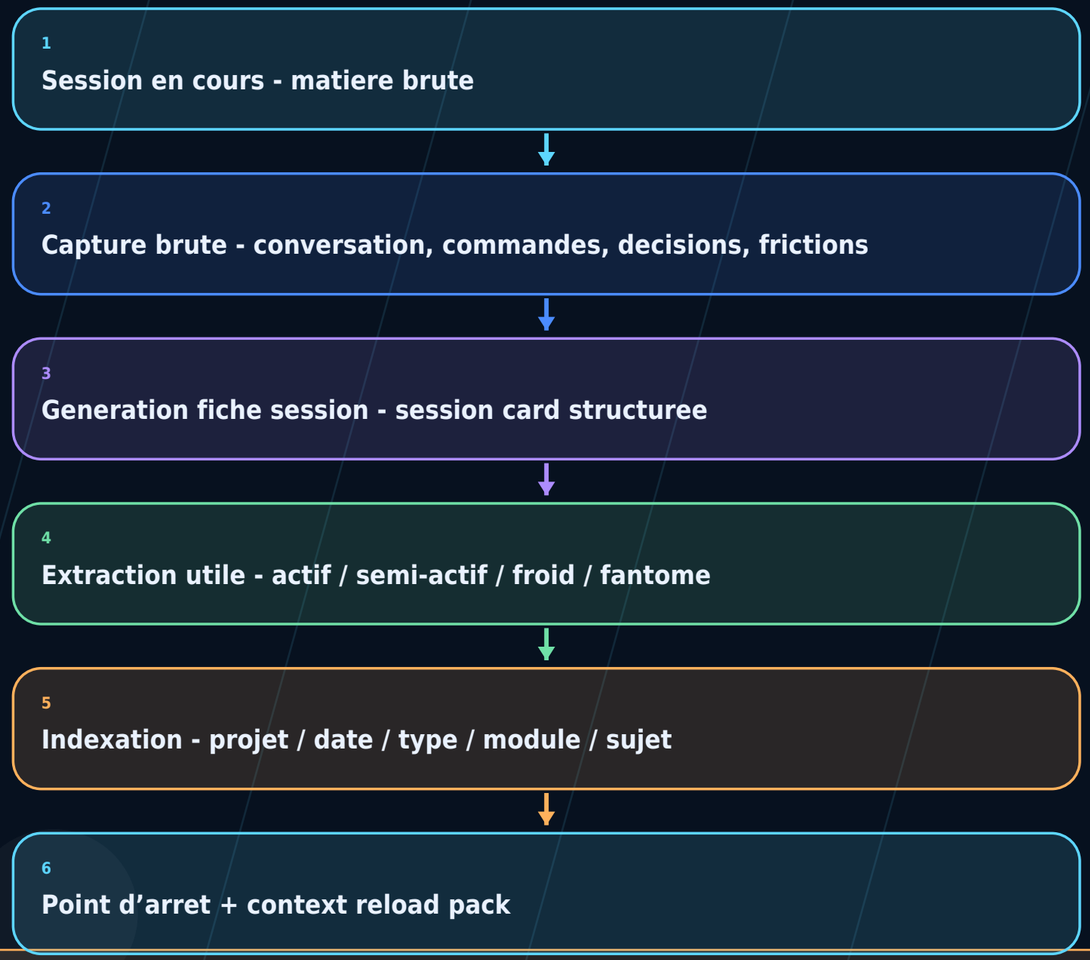
MESSAGE CLE

OBSIDIA CORE <-> SRL <-> UTILISATEURS / SESSIONS / WORKBENCH / AGENTS.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-04 - Flux complet SRL

Comment une session devient reprenable ?



MESSAGE CLE

A la reprise, le systeme recharge la fiche pertinente au lieu de relire toute la conversation brute.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-05 - Les 4 zones memoire

Actif, semi-actif, froid, fantome.

ACTIF

decision validee
point d’arret
module en cours
bloc a continuer
action prochaine

SEMI-ACTIF

idee secondaire
hypothese
piste non prioritaire
fragment lie a un projet

FROID

detail utile plus tard
ancienne piste
contexte secondaire
discussion non urgente

FANTOME

bruit possible
doublon
fragment faible
idee incertaine
element non prioritaire

MESSAGE CLE

Ne pas tout garder dans le contexte actif. Ne rien perdre. Tout classer. Tout rendre reactivable.

SRL-06 - Registre lateral des fragments ecartes

Meme ce qui est ecarte reste tracable.

01
ghost_registry.jsonl - registre des fragments fantomes

02
cold_index.csv - index froid

03
session_discard_table.json - table de mise de cote

04
fragment_id - identifiant fragment

05
session_id - source session

06
date - date capture

07
status - GHOST / COLD

08
reason - doublon / bruit / faible priorite / hors sujet temporaire

09
symbolic_hash - signature de sens

10
keywords - mots-cles de rappel

11
reactivation_policy - demande explicite / absence resultat / audit / collision semantique

MESSAGE CLE

Desindexe du flux principal, mais conserve dans une memoire laterale.

SRL-07 - Pseudo-hash symbolique

Retrouver un vieux bloc meme si les mots exacts changent.

EXEMPLE

SESSION:2026-06-04
TYPE:MEMORY_LAYER
DOMAIN:USER_SESSION
INTENT:CONTEXT_CONTINUITY
STATUS:COLD
GLYPH:NO_DELETE_SIDE_MEMORY

UTILITE

retrouver sans les mots exacts
reconnecter une idee ancienne
reactiver une pepite oubliee
auditer ce qui a ete mis de cote

MESSAGE CLE

HASH SYMBOLIQUE != hash cryptographique pur. HASH SYMBOLIQUE = signature de sens.

SRL-09 - Indexation multi-entrees

Chaque fiche doit pouvoir etre retrouvee par plusieurs entrees.

01
date

02
projet

03
type de session

04
module

05
concept

06
statut

07
point d’arret

08
pepite

09
decision

10
fichier

MESSAGE CLE

Exemples : 2026-06-04 / memoire session / SRL / peripherie / V0 ; ghost memory / cold storage / no delete.

SRL-10 - Reprise de session

Comment reprendre sans relire tout l’historique ?

1
Identifier le sujet actuel - detecter le contexte

2
Chercher les fiches liees - date / projet / type / module / concept

3
Charger seulement les fiches pertinentes - pas tout l’historique

4
Reprendre depuis exit_point - dernier point stable

5
Recuperer active_blocks - blocs utiles immediats

6
Ignorer cold / ghost sauf besoin - rappel profond conditionnel

REGLE STRICTE

READ_ALL_HISTORY = interdit par defaut
LOAD_SESSION_CARD = autorise
DEEP_RECALL = conditionnel

MESSAGE CLE

Une session ne doit pas etre relue. Elle doit etre retrouvee.

SRL-11 - Reactivation conditionnelle

Un fragment froid ou fantome revient seulement sous condition.

01

Demande explicite - l’utilisateur le demande

02

Memoire active insuffisante - la memoire active ne suffit pas

03

Audit - un audit exige de verifier

04

Collision semantique - un lien de sens apparait

05

Projet lie - un projet lie revient

06

Pepite ancienne - une pepite ancienne redevient utile

MESSAGE CLE

Le systeme ne fouille pas partout sans raison.

SRL-12 - Session Triage Agent

Le petit agent de tri prepare la continuite.

TRIAGE

classer / ranger

marquer / indexer

→

FROID

mettre en froid

reduire presence sans supprimer

→

FANTOME

mettre en fantome

hors contexte actif, retrouvable

→

PACK

fiche de reprise

context reload pack

BOUNDARIES

Il ne decide pas.
Il ne supprime pas.
Il ne modifie pas le noyau.
Il prepare la continuite.

MESSAGE CLE

Le Session Triage Agent prepare la continuite, pas l’autorite.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-13 - Limites V0

Le premier jet ne branche pas le noyau.

A NE PAS BRANCHER EN V0

memoire obsidienne centrale
Graphiti profond
X108 decisionnel
kernel
runtime moteur
lois internes completes

V0 RESTE SUR

fiche session
index
classement
reprise
memoire froide
memoire fantome

MESSAGE CLE

SRL V0 reste peripherique. Pas de noyau. Pas de decision. Pas de memoire centrale.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-14 - Branchage futur

Plus tard seulement, et sous conditions.



MESSAGE CLE

Le branchage futur devra respecter les boundaries.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-15 - Architecture V0 figee

Architecture figée de la couche sessionnelle.



MESSAGE CLE

Elle ne modifie pas Obsidia Core. Elle ne decide pas. Elle ne touche pas au noyau.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-16 - Branchement : historique de decision

Le document contient un cadrage puis une rectification.

HYPOTHESE INITIALE

SRL dans ObsidiaShell
peripherie
workbench
search
export
feature/srl-session-registry-layer-v0

RECTIFICATION

AUDIT CIBLE = Eaubin08/obsidia-x108-proofs UNIQUEMENT
PAS obsidiashell
PAS obsidia-engine-proof-core
PAS autre repo

MESSAGE CLE

Garder les deux comme historique de decision, sans les fusionner.

SRL-17 - Audit SRL dans obsidia-x108-proofs

SRL n’est pas a creer de zero.

ZONE CIBLE

periphery/brody_memory_readonly/
session_presave_buffer_readonly
project_intake_capture_buffer_readonly
apps/obsidia_api/brody_session_memory_adapter.py
apps/obsidia_api/brody_session_memory_runtime.py

VERDICT COURT

BASE SRL PRESENTE
COUCHE SRL NON CANONISEE
FRONTIERES READONLY A REALIGNER
METRIQUES SRL A AJOUTER

MESSAGE CLE

FOUNDATION_B_READY != SRL complete. FOUNDATION_B_READY = base existante pour SRL V0.

SRL-18A - Metriques ledger / follow-up / presave

Mesurer la base existante.

01
SRL_LEDGER_FOUND

02
SRL_LEDGER_RECORD_COUNT

03
SRL_JSONL_TRACE_VALID

04
SRL_SESSION_ID_PRESENT

05
SRL_PREVIOUS_EVENT_HASH_PRESENT

06
SRL_HASH_CHAIN_VALID

07
SRL_LAST_TURN_CONTEXT_FOUND

08
SRL_FOLLOWUP_SUPPORTED

09
SRL_FOLLOWUP_REQUESTED

10
SRL_FOLLOWUP_RESOLVED

11
SRL_TOPIC_EXTRACTED

12
SRL_LAST_CONTEXT_EXTRACTED

13
SRL_USED_BY_API_NOW

14
SRL_PRESAVE_POINTER_COUNT

15
SRL_POINTER_HASH_VALID

16
SRL_REFERENCED_PATH_COUNT

17
SRL_EVENT_HASH_CHAIN_VALID

18
SRL_MANUAL_VALIDATION_REQUIRED

MESSAGE CLE

Le script d’audit readonly doit lire et produire des rapports, sans rien modifier.

SRL-18B - Metriques triage / reprise

Prouver que le systeme ne remet pas tout dans le contexte actif.

01
SRL_ACTIVE_COUNT

02
SRL_SEMI_ACTIVE_COUNT

03
SRL_COLD_COUNT

04
SRL_GHOST_COUNT

05
SRL_ACTIVE_RATIO

06
SRL_COLD_RATIO

07
SRL_GHOST_RATIO

08
SRL_TRIAGE_CONFIDENCE

09
SRL_CONTEXT_OVERLOAD_SCORE

10
SRL_EXIT_POINT_FOUND

11
SRL_EXIT_POINT_QUALITY

12
SRL_RELOAD_PACK_CREATED

13
SRL_RELOAD_PACK_FRAGMENT_COUNT

14
SRL_RELOAD_PACK_MINIMALITY

15
SRL_DEEP_RECALL_USED

16
SRL_GHOST_REACTIVATION_COUNT

MESSAGE CLE

But : prouver que la session peut reprendre sans relire tout l’historique.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL-18C - Non-souverainete obligatoire

Contraintes obligatoires dans X108 Proof.

01
readonly = true

02
memory_write = false

03
graphiti_write = false

04
neo4j_write = false

05
memory_decision = false

06
allowed_to_decide = false

07
emits_act = false

08
emits_verdict = false

09
kernel_mutation = false

10
x108_runtime_binding = false

11
x108_merge = false

12
decision_authority = KX108_ONLY

MESSAGE CLE

SRL produit des context packets, pas des decisions.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

SRL - Plan V0 / V1

Audit, matrice, script readonly, extension readonly seulement si audit OK.

Phase	Livrables / contenu
Phase 1 - Audit existant	docs/runtime/OBSIDIA_SRL_SESSION_REGISTRY_LAYER_AUDIT_V0.md ; composants session existants ; fichiers ledger ; presave buffer ; follow-up resolver ; hash chain ; KX108_ONLY ; absence write/memory/Graphiti/X108 merge
Phase 2 - Matrice SRL	docs/runtime/OBSIDIA_SRL_COMPONENT_MATRIX_V0.json ; session_presave_buffer present ; session_ledger present ; followup_resolver partial ; active/semi/cold/ghost missing_or_partial ; symbolic_hash missing_or_partial ; context_reload_pack missing_or_partial ; deep_recall_policy missing_or_partial
Phase 3 - Script audit readonly	scripts/f_srl_session_registry_layer_audit_v0.py ; le script ne doit rien modifier ; produire MD + JSON
Phase 4 - Extension si audit OK	periphery/brody_memory_readonly/srl_session_registry_readonly/ ; srl_session_card_builder_readonly_v0.py ; srl_triage_metrics_readonly_v0.py ; srl_context_reload_pack_readonly_v0.py

MESSAGE CLE

SRL V0 = audit + formalisation + metriques + gaps. SRL V1 = extension readonly si l’audit confirme les manques.

SRL - Ce qu’il ne faut pas faire

Interdictions de branchement et mutation.

01

NE PAS toucher sigma/

02

NE PAS toucher proofs/lean/

03

NE PAS toucher formal/tla/

04

NE PAS toucher merkle_seal.json

05

NE PAS brancher Graphiti en ecriture

06

NE PAS faire x108_merge

07

NE PAS faire memory_intake=true

08

NE PAS faire allowed_to_decide=true

MESSAGE CLE

Nom freeze propre : OBSIDIA_SRL_SESSION_REGISTRY_LAYER_READONLY_AUDIT_V0.

Synthese finale - X108 vs SRL

Deux chemins separes : autorite et continuite.

Couche	Role	Peut decider ?	Peut ACT ?
Domaines	produire etat metier + signaux	NON	NON
Agents / AgentVote	observer, signaler, structurer	NON	NON
DomainAggregate	synthetiser le verdict metier candidat	NON	NON
Sigma	surveiller stabilite / proof signals	NON	NON
Kernel X108	autoriser, suspendre, bloquer	OUI	seule autorite ACT
OS3 / Audit	prouver, hasher, rejouer	NON	NON
SRL	rappeler, indexer, recharger contexte	NON	NON
Projection / UI	exposer lisiblement	NON	NON

MESSAGE CLE

Ne pas confondre autorite et continuite. X108 gouverne l’action. SRL gouverne la continuite contextuelle.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.

Carte finale fusionnee

Comprendre toute la fusion en une seule image.



MESSAGE CLE

Aucune couche peripherique ne peut court-circuiter X108.

Source redesign : reprise des details structurants de la version longue + approche schema-first. Non-runtime. Non-code.